

Vivado updatemem 流程的实验过程

2026-01-02

使用的双口 RAM 存代码，它的配置为 16x8k。布局布线后如下：

ram_16x8k_dp_pmem_shared/U0/inst_blk_mem_gen/gnbram.gnativebmg.native_blk_mem_gen/valid.cstr/ramloop[0].ram.r/prim_init.ram/DEVICE_7SERIES.NO_BMM_INFO.TRUE_DP.SIMPLE_PRIM36.ram	RAMB36E1	223
ram_16x8k_dp_pmem_shared/U0/inst_blk_mem_gen/gnbram.gnativebmg.native_blk_mem_gen/valid.cstr/ramloop[1].ram.r/prim_init.ram/DEVICE_7SERIES.NO_BMM_INFO.TRUE_DP.SIMPLE_PRIM36.ram	RAMB36E1	223
ram_16x8k_dp_pmem_shared/U0/inst_blk_mem_gen/gnbram.gnativebmg.native_blk_mem_gen/valid.cstr/ramloop[2].ram.r/prim_init.ram/DEVICE_7SERIES.NO_BMM_INFO.TRUE_DP.SIMPLE_PRIM36.ram	RAMB36E1	223
ram_16x8k_dp_pmem_shared/U0/inst_blk_mem_gen/gnbram.gnativebmg.native_blk_mem_gen/valid.cstr/ramloop[3].ram.r/prim_init.ram/DEVICE_7SERIES.NO_BMM_INFO.TRUE_DP.SIMPLE_PRIM36.ram	RAMB36E1	223

从上到下分别命名为 ramloop0,ramloop1,ramloop2 和 ramloop3。
怕截图看不清，再复制一下名字如下：

- ram_16x8k_dp_pmem_shared/U0/inst_blk_mem_gen/gnbram.gnativebmg.native_blk_mem_gen/valid.cstr/ramloop[0].ram.r/prim_init.ram/DEVICE_7SERIES.NO_BMM_INFO.TRUE_DP.SIMPLE_PRIM36.ram，物理位置 X4Y26
- ram_16x8k_dp_pmem_shared/U0/inst_blk_mem_gen/gnbram.gnativebmg.native_blk_mem_gen/valid.cstr/ramloop[1].ram.r/prim_init.ram/DEVICE_7SERIES.NO_BMM_INFO.TRUE_DP.SIMPLE_PRIM36.ram，物理位置 X4Y25
- ram_16x8k_dp_pmem_shared/U0/inst_blk_mem_gen/gnbram.gnativebmg.native_blk_mem_gen/valid.cstr/ramloop[2].ram.r/prim_init.ram/DEVICE_7SERIES.NO_BMM_INFO.TRUE_DP.SIMPLE_PRIM36.ram，物理位置 X3Y24
- ram_16x8k_dp_pmem_shared/U0/inst_blk_mem_gen/gnbram.gnativebmg.native_blk_mem_gen/valid.cstr/ramloop[3].ram.r/prim_init.ram/DEVICE_7SERIES.NO_BMM_INFO.TRUE_DP.SIMPLE_PRIM36.ram，物理位置 X3Y25

由于数据一直不对，所以代码里面增加了测试模式和 chipscope，去除写功能（不知道为什么 msp430 会对 pmem 进行写操作，这是另外一个要解决的问题），同时自己造地址读出整个 pmem,用 chipscope 看输出数据。为了便于检查，数据文件的开头，中间（4095 字节附近）和结尾的数据分别如下：

1	@0000
2	1234
3	5678
4	4031
5	0A00
6	430C
7	12B0
8	C044

4090	0000
4091	0000
4092	0000
4093	1122
4094	3344
4095	5566
4096	7788
4097	99aa
4098	bbcc
4099	ddee
4100	0000
4101	0000

8187	0000
8188	0000
8189	0000
8190	0000
8191	abcd
8192	1234
8193	C004

Mem 地址	数据	Mem 地址	数据	Mem 地址	数据
0x0000	0x1234	0x0ffb(4091)	0x1122	0x1ffd(8189)	0xabcd
0x0001	0x5678	0x0ffc(4092)	0x3344	0x1ffe(8190)	0x1234
0x0002	0x4031	0x0ffd(4093)	0x5566	0x1fff(8191)	0xc004
0x0004	0x0a00	0x0ffe(4094)	0x7788		
		0x0fff(4095)	0x99aa		
		0x1000(4096)	0xbbcc		

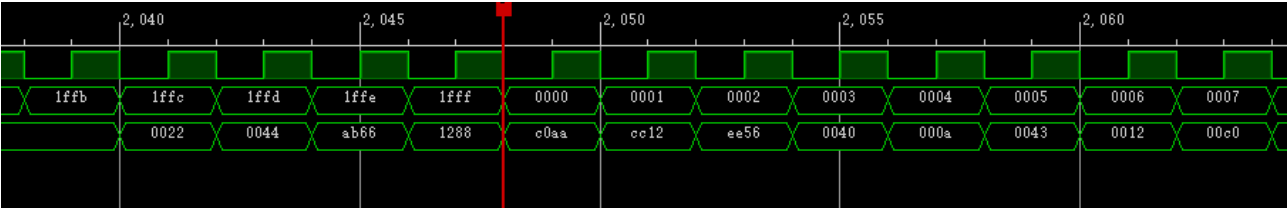
		0x1001(4097)	0xddee		

第一版的 mmi 文件如下（各种试验下来的结果）：

需要注意的是在 AddressSpace 这里面，后面的 Begin 和 End 都是以字节为单位。否则 updatemen 会一直报错说数据超过了地址范围。

```
<Processor Endianness="Little" InstPath="dummy">
  <AddressSpace Name="ram_16x8k_dp_pmem_shared/U0/inst_blk_mem_gen/gnbram.gnativebmg.native_blk_mem_gen" Begin="0" End="16383">
    <BusBlock >
      <BitLane MemType="RAMB32" Placement="X4Y26">
        <DataWidth MSB="7" LSB="0" />
        <AddressRange Begin="0" End="4095" />
        <Parity ON="false" NumBits="0" />
      </BitLane>
      <BitLane MemType="RAMB32" Placement="X4Y25">
        <DataWidth MSB="15" LSB="8" />
        <AddressRange Begin="0" End="4095" />
        <Parity ON="false" NumBits="0" />
      </BitLane>
    </BusBlock>
    <BusBlock >
      <BitLane MemType="RAMB32" Placement="X3Y25">
        <DataWidth MSB="7" LSB="0" />
        <AddressRange Begin="4096" End="8191" />
        <Parity ON="false" NumBits="0" />
      </BitLane>
      <BitLane MemType="RAMB32" Placement="X3Y24">
        <DataWidth MSB="15" LSB="8" />
        <AddressRange Begin="4096" End="8191" />
        <Parity ON="false" NumBits="0" />
      </BitLane>
    </BusBlock>
  </AddressSpace>
</Processor>
```

chipscope 抓到的波形如下：



上面三个信号从上到下分别为读使能，地址，数据。

再对着前面的输入数据表格看，会发现最后地址 0x1fff(8191)这一笔数据，预期的结果应该是 0xc004，但是结果是 0xc0aa,低 8 位的 0xaa 来自 0x0fff(4095)，也就是前一个 busblock 的低 8 位的最后一笔数据。0x1ffe(8190)的数据 0x1288，低 8 位的 0x88 来自 0x0ffe(4094)，也是来自前一个 busblock 的低 8 位。再看开头地址 0x0000 的数据，预期结果应该是 0x1234，但是变成了 cc12，显示高低字节反了，其次是 0xcc 来自后面的 busblock，地址为 0x1000(4096)的数据。

根据上面的线索，先把第一个 busblock 的 bitlane 高低字节调换，再把两个 buslock 的低 8 位互换位置。

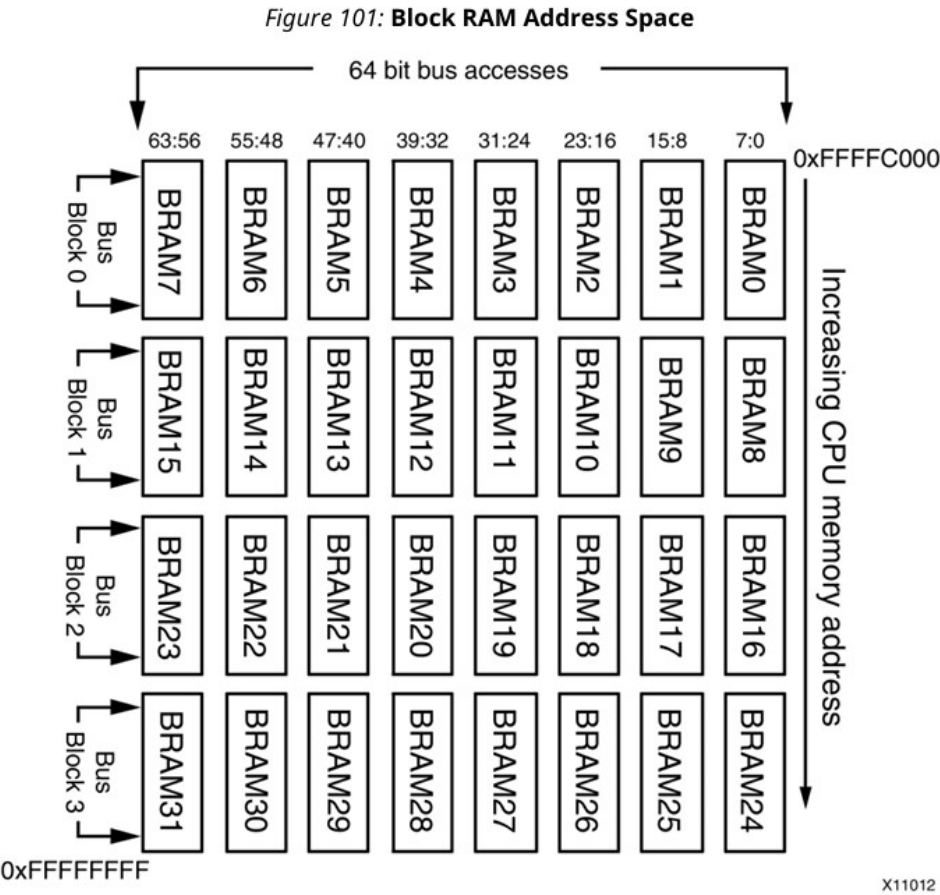
原来的状态：

地址 0, bit15			地址 0, bit0
	X4Y25 (1)	X4Y26 (0)	
	X3Y24 (2)	X3Y25 (3)	

很意外，按上面的思路互换完还是没有完全对，但是错误结果没记录。因为在开始、结束和 BusBlock 分界处放了几笔特征明显的数字，看着波形调整，很快就得到最终正确的结果如下：

地址 0, bit15			地址 0, bit0
	X4Y26 (0)	X3Y24 (2)	
	X4Y25 (1)	X3Y25 (3)	

这个结果，和 ug898 里面的这个图不太能对上：



总结:

1. `updatemem` 对于需要固化数据到 rom 中设计确实有很大的优势。

```
#updatemem --force -meminfo ../scripts/memory.mmi -data $1.mem -proc dummy -bit ./openMSP430_fpga.bit -out $1.bit
updatemem --force -meminfo ../scripts/memory2.mmi -data leds2.mem -proc dummy -bit ./openMSP430_fpga.bit -out $1.bit
# Copy new bitstream in the proper directory
```

这个命令执行很快，吃进一个 `mmi`，一个 `mem` 和一个 `bit` 文件，生成一个新的 `bit` 文件，里面的 `rom` 和 `ram` 就填好了数据。意味着我们的固件、OSD 素材、配置等等，如果最终需要保存到 `blockram` 中，都可以用这个方法。

2. 难点就是这个 `blockram` 被 `vivado` 自动拆分成几个更小的 `ramb36` 或者 `ramb18` 后，怎么确定摆放位置。因为每次综合后它们的物理位置会发生变化。按照我现在的位置，我需要添加好 `ChipScope` 和 `Debug` 的代码，跑完后去 `vivado gui` 中打开 `impl` 的设计，`Ctrl+F` 查找所有的 `blockram`，把坐标抄下来，再造一个 `mem` 文件，在头、尾和 `BusBlock` 分界的地方放上几笔特殊数据，再一次一次的试。我能想到的好办法是碰到这些要用 `updatemem` 更新数据的 `ram` 或者 `rom`，手工拆成几个小的，确保每一个小的 `ram` 只用一个 `ramb36` 或者 `ramb18`，再给它们取好名字。实现完成后通过脚本 `report` 它们的坐标，根据前面实验的结果自动生成 `mmi` 文件。这一部分有待实验。